

C++ Built-in Functions

ECPC Competitive Programming Reference

Common algorithms, conversions, and utilities from the C++ Standard Library

1 Type Conversions & String Utilities

Function	Purpose	Complexity
<code>to_string(x)</code>	Convert numeric value (int, long long, float, double, ...) to <code>string</code>	$O(\#digits)$
<code>stoi(s)</code> / <code>stoll(s)</code>	Convert string of digits to <code>int</code> / <code>long long</code>	$O(s)$
<code>stof(s)</code> / <code>stod(s)</code>	Convert string to <code>float</code> / <code>double</code>	$O(s)$
<code>stringstream</code>	Stream-based conversion, parsing, or tokenizing space-separated tokens	$O(s)$
<code>atoi(cstr)</code>	Convert C-style string (<code>const char*</code>) to <code>int</code>	$O(s)$
<code>c - '0'</code>	Convert digit character '0'–'9' to integer value	$O(1)$
<code>d + '0'</code>	Convert integer digit 0–9 to corresponding character	$O(1)$

```
int number = 1337;
string str_number = to_string(number);           // "1337"

string str_int = "42";
int num = stoi(str_int);                         // 42

string str_double = "3.14159";
double pi = stod(str_double);                   // 3.14159

string text = "100 200 300";
stringstream ss(text);
int a, b, c; ss >> a >> b >> c;                  // a=100, b=200, c=300

const char* cstr = "999";
int c_num = atoi(cstr);                         // 999

char digit_char = '7';
int digit_int = digit_char - '0';                // 7

int val = 5;
char val_char = val + '0';                      // '5'
```

2 Searching & Boundaries

Binary operations require a sorted range.

Function	Purpose	Complexity
<code>binary_search</code>	Returns <code>true</code> if value exists in sorted range	$O(\log N)$
<code>lower_bound</code>	Iterator to first element $\geq$ value	$O(\log N)$
<code>upper_bound</code>	Iterator to first element $>$ value	$O(\log N)$
<code>find</code>	Linear search for first occurrence of value	$O(N)$
<code>search</code>	Find first occurrence of a subsequence (pattern)	$O(N \cdot M)$

```
vector<int> sorted_v = {10, 20, 30, 30, 30, 40, 50};

bool exists = binary_search(sorted_v.begin(), sorted_v.end(), 30); // true

auto lb = lower_bound(sorted_v.begin(), sorted_v.end(), 30);
// index = lb - sorted_v.begin() -> first 30

auto ub = upper_bound(sorted_v.begin(), sorted_v.end(), 30);
// index = ub - sorted_v.begin() -> 40

auto it = find(sorted_v.begin(), sorted_v.end(), 40);

vector<int> pattern = {30, 30};
auto sit = search(sorted_v.begin(), sorted_v.end(),
                  pattern.begin(), pattern.end());
```

3 Sorting & Rearranging

Function	Purpose	Complexity
<code>sort</code>	Sort range ascending (or with custom comparator)	$O(N \log N)$
<code>sort + greater<></code>	Sort range descending	$O(N \log N)$
<code>reverse</code>	Reverse order of elements	$O(N)$
<code>rotate</code>	Rotate so that middle becomes the new first element	$O(N)$

```
vector<int> v = {5, 2, 9, 1, 5, 6};

sort(v.begin(), v.end());           // 1 2 5 5 6 9
sort(v.begin(), v.end(), greater<int>()); // 9 6 5 5 2 1

reverse(v.begin(), v.end());        // reverse in-place
rotate(v.begin(), v.begin() + 2, v.end()); // middle at index 2
```

4 Modifying & Cleaning

Function	Purpose	Complexity
<code>unique + erase</code>	Remove consecutive duplicates (sort first to remove all)	$O(N)$
<code>fill</code>	Fill range with a specified value	$O(N)$
<code>insert</code>	Insert element/range at position (shifts elements)	$O(N)$ (vector)

```
vector<int> dup = {1, 2, 2, 3, 3, 3, 4, 1};
sort(dup.begin(), dup.end());
dup.erase(unique(dup.begin(), dup.end()), dup.end()); // 1 2 3 4

vector<int> fill_v(5);
fill(fill_v.begin(), fill_v.end(), -1);           // -1 -1 -1 -1 -1

fill_v.insert(fill_v.begin() + 2, 99);             // insert 99 at index 2
```

5 Permutations

Function	Purpose	Complexity
<code>next_permutation</code>	Next lexicographical permutation; returns <code>false</code> at last	$O(N)$ per call

To generate all permutations: start from sorted order and loop until `next_permutation` returns `false`. Total time $O(N \cdot N!)$.

```
vector<int> perm = {1, 2, 3};
do {
    // process current permutation
} while (next_permutation(perm.begin(), perm.end()));
```

6 Min / Max Operations

Function	Purpose	Complexity
<code>min / max</code>	Minimum / maximum of two values (or initializer list)	$O(1)$ / $O(k)$
<code>min_element</code>	Iterator to smallest element in range	$O(N)$
<code>max_element</code>	Iterator to largest element in range	$O(N)$
<code>minmax_element</code>	Pair of iterators (min, max) in a single pass	$O(N)$

```
int x = 15, y = 42, z = 7;
cout << min(x, y);           // 15
cout << max({x, y, z});      // 42

vector<int> arr = {14, 2, 88, 45, 2};
auto min_it = min_element(arr.begin(), arr.end());
auto max_it = max_element(arr.begin(), arr.end());
auto p = minmax_element(arr.begin(), arr.end()); // {min, max}
```

7 Numeric Algorithms (<numeric>)

Function	Purpose	Complexity
<code>accumulate</code>	Sum of range + initial value (use 0LL for long long)	$O(N)$
<code>gcd</code> / <code>lcm</code>	Greatest Common Divisor / Least Common Multiple	$O(\log \min(a, b))$
<code>iota</code>	Fill range with consecutive values starting from given value	$O(N)$

```
vector<int> nums = {10, 20, 30, 40};
long long total = accumulate(nums.begin(), nums.end(), 0LL); // 100

cout << gcd(12, 18);           // 6
cout << lcm(12, 18);           // 36

vector<int> seq(5);
iota(seq.begin(), seq.end(), 1); // 1 2 3 4 5
```

8 Counting & Conditions

Function	Purpose	Complexity
<code>count</code>	Number of elements equal to a given value	$O(N)$
<code>count_if</code>	Number of elements satisfying a predicate (lambda)	$O(N)$
<code>all_of</code>	true if all elements satisfy the condition	$O(N)$
<code>any_of</code>	true if at least one element satisfies the condition	$O(N)$

```
vector<int> data = {1, 2, 2, 3, 4, 2, 5};

int twos = count(data.begin(), data.end(), 2); // 3

int evens = count_if(data.begin(), data.end(),
    [](int v){ return v % 2 == 0; }); // 3

bool all_pos = all_of(data.begin(), data.end(),
    [](int v){ return v > 0; }); // true

bool any_gt4 = any_of(data.begin(), data.end(),
    [](int v){ return v > 4; }); // true
```

Headers required: <bits/stdc++.h> (or specific headers such as <algorithm>, <numeric>, <string>, <sstream>).
All binary-search based algorithms require the range to be sorted.